

Building a Flight Price Comparison Tool with Claude Code

Sparsh Kapoor — March 23, 2026

Table of Contents

1. [The Idea](#)
 2. [Repo Building](#)
 3. [Usage Limits](#)
 4. [Is Everything Correct?](#)
 5. [What's Next](#)
-

The Idea

Every time I book a flight, I do the same thing. I open Google Flights, search for a round trip, note the price, then go back and search the outbound and return separately as one-ways. Sometimes the two one-ways are cheaper. Sometimes staying on the same airline for both legs saves money. No tool surfaces this comparison directly — you have to do the math yourself.

FlightSelect automates that. You enter your origin, destination, and dates, and it scrapes real Google Flights pricing via SerpAPI, runs both combinations, and tells you: **Same Airline** (cheapest round trip on one carrier) vs. **Best Mix** (cheapest outbound + cheapest return, any carrier). It shows you the savings and links you to Google Flights to book.

I built the whole thing with [Claude Code](#) — Anthropic's CLI agent for software engineering. Every feature, every bug fix, every architectural decision was a conversation in the terminal. This post walks through how that went.

The Stack

- **Frontend:** React 18, Vite, TailwindCSS, Zustand, React Query
 - **Backend:** Express, Prisma ORM, PostgreSQL, Redis, BullMQ
 - **Shared:** TypeScript monorepo with npm workspaces, Zod schemas
 - **Data:** SerpAPI Google Flights API (free tier: 250 searches/month)
-

Repo Building

Starting Point

The repo started as a GitHub Copilot-generated starter project. It had the monorepo scaffolding, a Prisma schema, a React frontend with a search form, and placeholder scraper interfaces. It looked complete at a glance, but nothing actually worked.

I opened Claude Code and pointed it at the repo. The first session was entirely about getting the thing to run.

Infrastructure Issues

Docker Compose failed immediately. Port 5432 was already in use by my local Postgres installation. Claude Code found the conflict (needed `sudo lsof` to see it — regular `lsof` returned nothing) and remapped the Docker container to port 5433.

Then the database migrations failed because the Docker containers hadn't actually started. Then the seed script failed with TypeScript errors — `CabinClass` was imported from `@prisma/client` in some places and `@flightselect/shared` in others. Both enums had identical string values, but TypeScript didn't care. Claude fixed the imports and discovered the critical build order: shared package first, then Prisma migration, then seed.

The Missing Pieces

The server crashed on startup because `dotenv` was never installed. Environment variables were validated with Zod, but nothing loaded the `.env` file first. A one-line fix — `import 'dotenv/config'` as the first import in `index.ts` — but it had to be *first*. In CommonJS, import order matters.

The bigger problem: searches were submitted successfully (I could see the `SearchQuery` in the database and the job in Redis) but no flights ever appeared. The BullMQ worker function existed in the codebase — `createSearchWorker()` — but was never called. Jobs were queued and sat there. Claude added the worker startup to `bootstrap()` and flights started flowing.

```
// This function existed but was never invoked anywhere
await createSearchWorker((job) =>
  processSearchJob(job.data as { searchQueryId: string })
);
```

The Filter UI

The filter store defined 8 filter fields. The FilterSidebar rendered 3. Claude added the missing ones — flight duration slider, departure time range — and wired up the actual filter logic in the results page. The filtering is all client-side: fetch all flights once, filter in the browser for instant responsiveness.

By the end of the first session, the app ran end-to-end on mock data. You could search, see flights, filter results, and view the comparison analysis. But it was all fake — randomly generated prices and airlines.

Usage Limits

Before touching real flight data, I needed to think about rate limiting. SerpAPI gives you 250 searches per month on the free tier. Each search in FlightSelect makes **two** API calls (outbound one-way + return one-way), so that's effectively 125 searches per month. And since this was going on the internet for a blog post, anyone could hit the endpoint.

I asked Claude Code to implement a two-layer rate limiter:

Per-Client Sliding Window

Each client (identified by IP) gets 1 search per 60 seconds. The implementation uses a Redis key with a 60-second TTL:

```
const clientKey = `ratelimit:serpapi:client:${clientIp}`;
const exists = await redis.exists(clientKey);

if (exists) {
  const ttl = await redis.ttl(clientKey);
  return res.status(429).json({
    error: `Rate limited: please wait ${ttl} seconds before searching again.`,
    retryAfterSeconds: ttl,
    limitType: 'per_client',
  });
}

await redis.set(clientKey, '1', 'EX', 60);
```

If the key exists, you're blocked. If not, it's created with a 60-second expiry and your request goes through. Simple, but it prevents any single user from burning through the quota.

Global Monthly Cap

A second Redis counter tracks total searches across all users for the billing cycle. SerpAPI renews on the 28th of each month, so the key is computed dynamically:

```
function getBillingCycleKey(renewalDay: number): string {
  const now = new Date();
  const year = now.getFullYear();
  const month = now.getDate() >= renewalDay
    ? now.getMonth() + 1
    : now.getMonth();
  return `ratelimit:serpapi:global:${year}-${String(month).padStart(2, '0')}`;
}
```

I set the cap at 225 (leaving 25 as a buffer for testing). When the counter hits 225, every user sees a 429 with `limitType: 'global_monthly'` — no more searches until the next billing cycle.

Fail-Open Design

If Redis goes down, the rate limiter lets requests through. A Redis outage shouldn't break the entire app — you'd rather risk a few extra API calls than have the site go completely unresponsive.

Is Everything Correct?

I wrote a test script to verify the rate limiter actually works. It's not a unit test — it hits the real API endpoint and checks the responses.

```
npx ts-node scripts/test-rate-limit.ts
```

The test suite checks:

1. **First request succeeds** (202 Accepted)
2. **Same client blocked within 60s** (429 with `per_client` type)
3. **Rate limit status endpoint** returns remaining quota
4. **Spam protection** — fires 5 concurrent requests, verifies at most 1 succeeds
5. **Response headers** include `X-RateLimit-Remaining` and `X-RateLimit-Limit`
6. **Global cap** (optional) — sets the Redis counter to 225, confirms 429, then restores the original value

The spam test was the one I cared about most. Five concurrent `POST /api/search` requests from the same client — the Redis `SET ... NX EX` ensures only one gets through. The others all get 429s.

Beyond rate limiting, I had Claude fix several issues that would have been embarrassing in a blog post:

- **TypeScript errors in the mock scraper** — `cabinClass: string` didn't match the `CabinClass` enum type. Non-blocking since the mock was disabled, but still a red squiggle I didn't want visible.
- **Prisma query logging** was dumping every SQL statement to the console. Changed `log: ['query', 'error', 'warn']` to `log: ['error', 'warn']`.
- **Frontend TS error** — `data?.flights?.length > 0` where `undefined > 0` is not a valid comparison. Fixed with a nullish coalescing operator.
- **Airport autocomplete** — typing "LOS" auto-filled "Lagos" before you could finish typing "Los Angeles." Rewrote the input to use a dropdown with relevance-ranked search instead of auto-fill. Exact code matches rank highest, then starts-with, then contains.
- **Filters were broken** — the layover filter only checked a boolean `isLayover`, not the actual layover count. Price slider reset on every re-render. Added a sort dropdown (price, duration, departure time).
- **Layover airports were always null** — the scraper wasn't extracting layover data from SerpAPI's segment arrays correctly. Added a fallback parser.

What's Next

With the infrastructure solid and the UI cleaned up, the next step was plugging in real data. The SerpAPI Google Flights scraper had a stub in the codebase but it was completely broken — wrong parameter types, wrong date formats, missing the `type` field entirely.

Claude fixed six bugs in the scraper in one session:

- `travel_class` needed to be numeric (1/2/3/4), not strings like "economy"
- Dates were sent as `Date` objects instead of `YYYY-MM-DD` strings
- Round-trip mode in SerpAPI requires a two-step `departure_token` flow — we switched to two separate one-way calls instead
- Flights with no price came back from the API with `undefined`, and `undefined * passengers = NaN`, which crashed Prisma's `createMany`

Once the scraper worked, the comparison engine needed a rewrite. The old version compared "Round Trip" vs "One-Way" but both sides showed the same flights with a fabricated 10% discount. The new engine does what the app was always supposed to do:

- **Same Airline:** iterate every outbound flight, find the cheapest return flight on the same carrier, take the cheapest combo
- **Best Mix:** cheapest outbound + cheapest return, regardless of airline

All prices are real, sourced from Google Flights via SerpAPI.

Going forward, there are two features I want to build:

1. **RAG pipeline** — scrape and persist historical flight pricing data so an LLM (likely Ollama running locally) can surface patterns like "this route is usually \$50 cheaper on Tuesdays" or "prices for this destination typically drop 3 weeks before departure."
2. **Direct airline booking links** — SerpAPI returns a `booking_token` on each flight that can be used to fetch actual airline booking URLs. Right now we link to a Google Flights search page; eventually we'd link directly to United, Delta, etc.

The entire project — from broken starter repo to working flight comparison tool with real data, rate limiting, and a clean UI — was built through conversations with Claude Code. Every bug fix, every architectural decision, every line of code.

The repo is at github.com/sparshkapoor/flightselect.